



8



19



2280 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

server-side
development

د. غاندي هسام

Programming
Languages



RB Informatics; 3/12/2025

لغات البرمجة

الواجهة الأمامية (front End) مقابل الواجهة الخلفية (Back End):

الواجهة الأمامية (Front End - ما يراه المستخدم):

هي كل ما يعمل داخل المتصفح (Browser).

- HTML: هيكل الصفحة.
- CSS: الشكل والتصميم (الألوان، الخطوط).
- JavaScript: التفاعل والمنطق داخل الصفحة.

الواجهة الخلفية (Back End - ما يحدث خلف الكواليس):

هي الكود الذي يعمل على الخادم (Server).

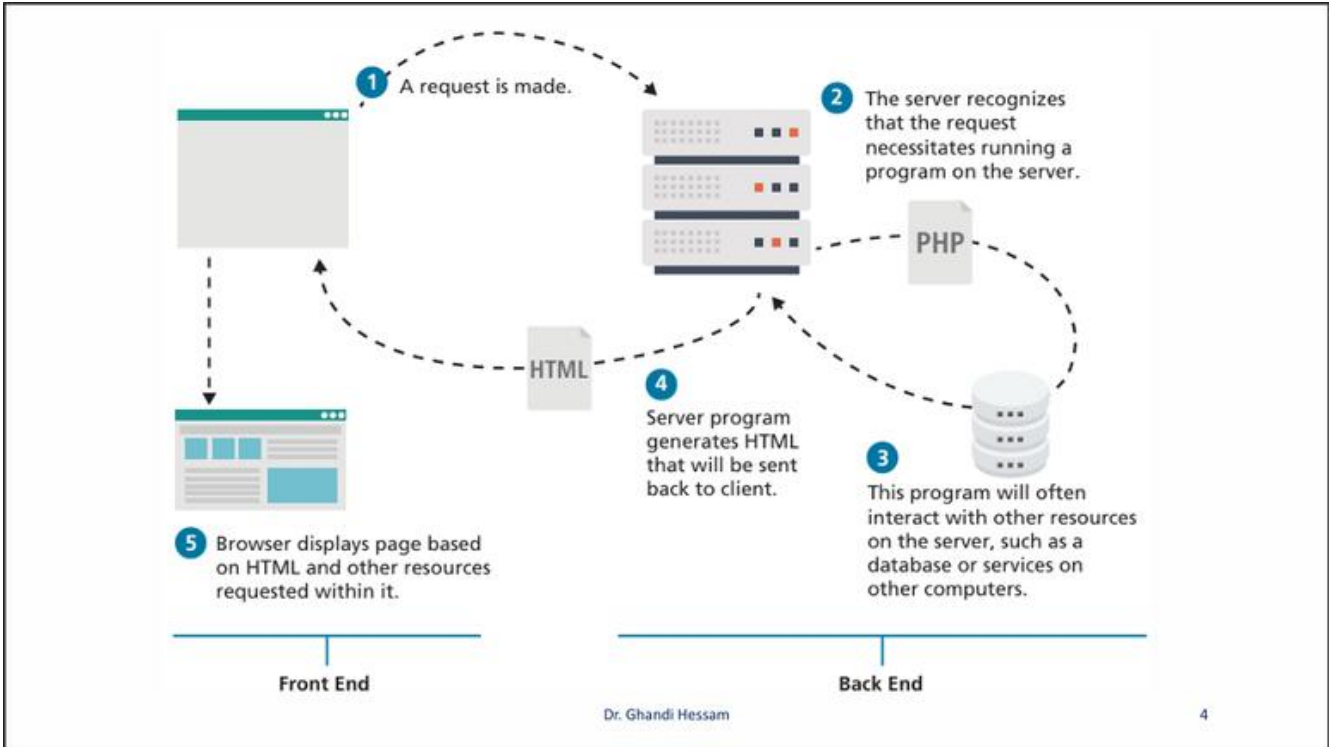
مسؤول عن "العمليات الثقيلة": جلب البيانات من قواعد البيانات، حماية الموقع (Security)، والربط مع خدمات أخرى (مثل البنوك للدفع).

كيف تطور الوضع؟

قديمًا (Traditional): كان السيرفر يقوم بكل شيء، يجهز الصفحة كاملة (HTML) ويرسلها جاهزة للمتصفح.

حديثًا (Modern/SPA): أصبح المتصفح ذكيًا وقويًا. السيرفر يرسل فقط "البيانات الخام" (Data via APIs)، والمتصفح (Front End) هو من يقوم بإنشاء وتشكيل الصفحة وعرض البيانات.





أساسيات لغة PHP:

- المرونة في أنواع البيانات: تماماً كما في JavaScript، هذا يعني أن المتغير يمكن أن يكون رقماً، ثم يتحول لاحقاً إلى نص (String)، ثم لاحقاً إلى كائن (Object).
- دعم البرمجة كائنية التوجه: توفر PHP الفئات (Classes) والدوال (Functions) بطريقة تتوافق (تتشابه) مع لغات البرمجة كائنية التوجه (OOP) الأخرى.
- تشابه النحو (Syntax): طريقة كتابة الحلقات التكرارية (Loops)، والجمل الشرطية (Conditionals)، وعمليات الإسناد (Assignment) مطابقة لـ JavaScript. الاختلاف يظهر فقط عند كتابة الدوال، والفئات، وطريقة تعريف المتغيرات.
- الميزة الأهم: الحقيقة الأكثر أهمية حول PHP هي أن الكود البرمجي يمكن تضمينه (Embedded) مباشرة داخل ملف HTML.
- آلية العمل: يتم تفسير وتنفيذ الكود البرمجي الموجود داخل وسوم `<?php ... ?>`، بينما الكود الموجود خارج هذه الوسوم يتم إرساله مباشرة كما هو إلى العميل (المتصفح).

كيف تعمل PHP؟

- ملفات PHP هي في الأصل ملفات HTML عادية، لكننا نفتح داخلها "نافذة صغيرة" للكود البرمجي باستخدام الوسم `<?php>`.
- عندما يطلب المستخدم الصفحة، يقرأ السيرفر الملف.

- أي شيء داخل <? php?> ينفذه السيرفر (عمليات حسابية، جلب بيانات).
- أي شيء خارج هذه الوسوم (HTML عادي) يرسله السيرفر للمستخدم كما هو.
- النتيجة: المستخدم يرى صفحة HTML صافية، ولا يرى كود PHP أبداً.

Example

On the server (example.php):

```
<?php
$user = "Randy";
?>
<!DOCTYPE html>
<html>
<body>
<h1>Welcome <?php echo $user; ?></h1>
<p>
The server time is
<?php
echo "<strong>";
echo date("H:i:s");
echo "</strong>";
?>
</p>
</body>
</html>
```

What the client receives:

```
<!DOCTYPE html>
<html>
<body>
<h1>Welcome Randy</h1>
<p>
The server time is <strong>02:59:09</strong>
</p>
</body>
</html>
```

Dr. Ghandi Hssam

6

المتغيرات والثوابت في PHP:

- أنواع البيانات المرنة (Loosely Typed):
المتغيرات أيضاً تعتبر "مرنة النوع" (Loosely Typed)، بمعنى أنه يمكن إسناد أنواع بيانات مختلفة لنفس المتغير بمرور الوقت.

○ تعريف المتغيرات:

للتصريح عن متغير (Declare a variable)، يجب أن تسبق اسم المتغير برمز الدولار (\$).

○ الثوابت (Constants):

يمكن تعريف الثابت في أي مكان، ولكن عادةً ما يتم تعريفه بالقرب من أعلى ملف PHP باستخدام الدالة define().

```
<?php

// uppercase for constants is a programming convention
define("DATABASE_LOCAL", "localhost");
define("DATABASE_NAME", "ArtStore");
define("DATABASE_USER", "Fred");
define("DATABASE_PASSWD", "F5^7%ad");
// ...
// notice that no $ prefaces constant names
$db = new mysqli(DATABASE_LOCAL, DATABASE_NAME, DATABASE_USER,
    DATABASE_PASSWD);

?>
```

Program Control

```
<?php if ($userStatus == "loggedin") { ?>
    <a href="account.php">Account</a>
    <a href="logout.php">Logout</a>
<?php } else { ?>
    <a href="login.php">Login</a>
    <a href="register.php">Register</a>
<?php } ?>

<?php
// equivalent to the above conditional
if ($userStatus == "loggedin") {
    echo '<a href="account.php">Account</a> ';
    echo '<a href="logout.php">Logout</a>';
}
else {
    echo '<a href="login.php">Login</a> ';
    echo '<a href="register.php">Register</a>';
}
?>
```

```
<?php if ($userStatus == "loggedin") : ?>
    <a href="account.php">Account</a>
    <a href="logout.php">Logout</a>
<?php else : ?>
    <a href="login.php">Login</a>
    <a href="register.php">Register</a>
<?php endif; ?>
```

LISTING 12.12 Alternate syntax for control structures

Dr. Ghandi Hessam

10

طرق التحكم في PHP:

1. طريقة "الفتح والإغلاق" (الموجودة أعلى اليسار):

- الفكرة: نفتح PHP فقط لكتابة سطر الشرط if والقوس {}, ثم نغلقه فوراً.
- العمل: نكتب كود HTML (الروابط) بشكل عادي خارج PHP.
- الإغلاق: نعود ونفتح PHP مرة أخرى فقط لنكتب قوس الإغلاق {}.
- باختصار: نقطع كود PHP لنحشر كود HTML في الوسط.

2. طريقة "الطباعة بـ echo" (الموجودة أسفل اليسار):

- الفكرة: كل شيء يبقى داخل صندوق PHP واحد.
- العمل: لا نكتب HTML عادي، بل نستخدم أمر echo لطباعة الروابط كنصوص.
- باختصار: كود HTML مخبأ داخل كود PHP.

3. طريقة "النقطتين و endif" (الموجودة على اليمين - وهي الأهم):

- الفكرة: نستبدل الأقواس المعقوفة {} بكلمات واضحة للقراءة.
- البديل:

▪ بدل القوس { } نضع نقطتين رأسيين ::

▪ بدل القوس { } نكتب كلمة endif في النهاية.

- ميزة فريدة: تمتلك PHP ميزة مهمة جداً تختلف بها عن معظم لغات البرمجة الأخرى (التي لا تعمل للويب).

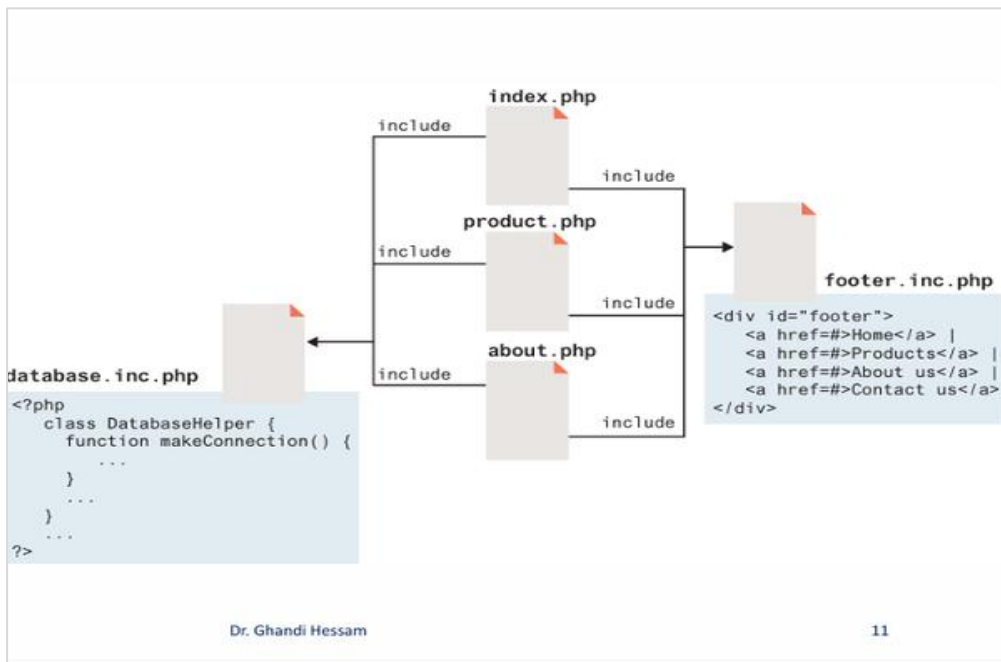
- الميزة هي: القدرة على تضمين (Include) أو إدخال (Insert) محتوى موجود في ملف ما، داخل ملف آخر.

تخيل أن لديك موقعاً يحتوي على 100 صفحة. كل صفحة في الموقع لها نفس الرأس (Header) ونفس تذييل الصفحة (Footer).

- بدون هذه الميزة: ستضطر لنسخ كود القائمة (Menu) وكود التذييل في كل الـ 100 ملف. وإذا أردت تغيير كلمة واحدة، ستعدلها 100 مرة!
- مع ميزة Include:

- تكتب كود القائمة مرة واحدة في ملف اسمه header.php.
- تكتب كود التذييل مرة واحدة في ملف اسمه footer.php.
- في كل صفحة من صفحات الموقع، تكتب فقط سطرًا واحدًا: `'include 'header.php`.

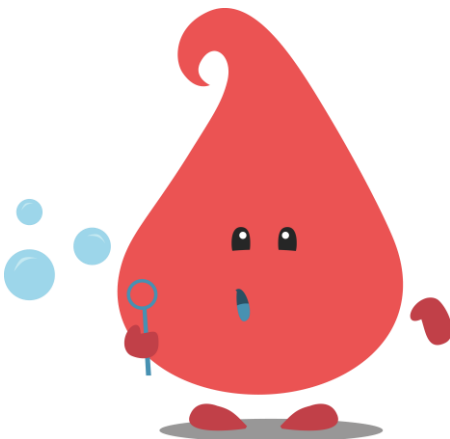
الفكرة ببساطة: هي مثل عملية "قص ولصق" (Copy/Paste) لكنها تحدث أوتوماتيكياً بواسطة السيرفر عند طلب الصفحة.



تضمين الملفات في PHP:

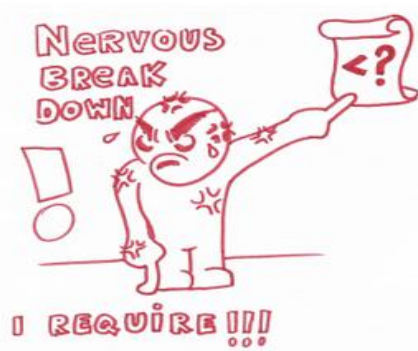
توفر PHP أربع جمل مختلفة لتضمين الملفات، كما هو موضح في المثال التالي:

- `include "somefile.php"`
- `include_once "somefile.php"`
- `require "somefile.php"`
- `require_once "somefile.php"`



ما الفرق بينهم؟

- الفرق في الأخطاء: مع include، يتم عرض تحذير (Warning) ويستمر تنفيذ البرنامج. أما مع require، يتم عرض خطأ (Error) ويتوقف التنفيذ فوراً.
- منع التكرار: عبارتا include_once و require_once تعملان تماماً مثل include و require، ولكن إذا كان الملف المطلوب قد تم تضمينه مرة واحدة سابقاً، فلن يتم تضمينه مرة أخرى.



When Mr. Php Script "requires" he makes a nervous break down and **crashes**, if not found.



In the other hand, when Mr. Php Script is asked to "include" the file, he just apologizes for inconvenience with a little "Warning" not found.

By convention, PHP files have the .php extension.

Files that are included can have any extension, though in this example we are using the extension .inc.php to make it clearer later that this is an include file.

example.php

```
<?php
include('exampleData.inc.php');
?>
<!DOCTYPE html>
<html lang="en">
<head>
...
</head>
<body>
<form>
  <fieldset>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" value="<?= $name ?>" >
```

exampleData.inc.php

```
<?php
$name = 'Randy Connolly';
$email = 'someone@example.com';
?>
```

The include function inserts the contents of the specified file.

Common practice is to place include statements (and variables used throughout the page) at the top of the page.

Here we are outputting the contents of the \$name variable into the value attribute.

Dr. Ghandi Hssam

13

هذه الصورة تشرح مبدأ "التضمين" (Include) لترتيب الكود.

اللف الرمادي (exampleData.inc.php):

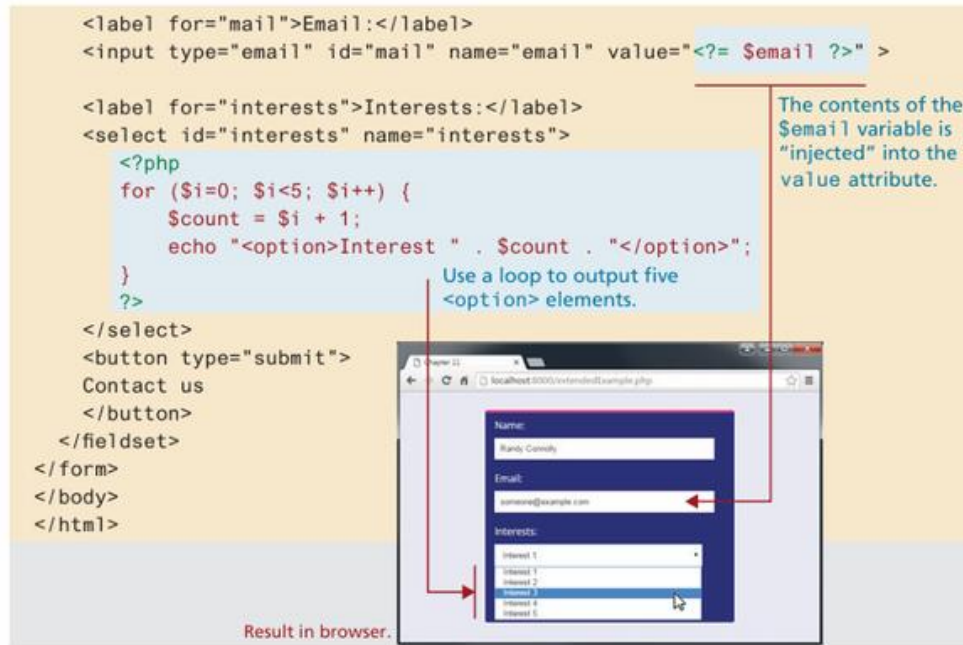
- هذا الملف وظيفته "مخزن بيانات" فقط.
- يحتوي على متغيرات: الاسم \$name والإيميل \$email.

- امتداده inc.php. لنميزه أنه ملف "مُصَنَّع" وليس صفحة كاملة.

🔗 الملف البرتقالي (example.php):

- هذه هي الصفحة الحقيقية التي يراها المستخدم.
- في الأعلى: استخدمنا include لجلب البيانات من الملف الرمادي.
- في الأسفل (داخل النموذج): استخدمنا الكود `<? name$ = ?>` لطباعة الاسم داخل الحقل (Input).

الخلاصة: بدل ما نكتب الاسم "Randy" يدوياً في كود HTML، وضعناه في متغير واستدعيناه. هذا يسهل التعديل لاحقاً.



Dr. Ghandi Hssam

14

هذه الصورة تعرض مثلاً أكثر تطوراً: كيف نملاً القوائم (Dropdowns) وحقول الإدخال أوتوماتيكياً باستخدام PHP.

🔗 الجزء العلوي (حقل الإيميل):

- الكود: `<? value="<? $email ?>">`
- الشرح: هنا "حقنا" (Injected) قيمة المتغير `$email` مباشرة داخل الحقل. يعني المستخدم سيفتح الصفحة ويجد إيميله مكتوباً جاهزاً.

🔗 الجزء السفلي (قائمة الاهتمامات - Interests):

- هنا نرى قوة PHP الحقيقية داخل وسم `<select>`.
- بدل كتابة الخيارات `<option>` خمس مرات يدوياً، كتبنا حلقة تكرار (For Loop).
- الكود: `for ($i=0; $i<5; $i++)`
- ماذا يفعل؟ يدور 5 مرات، وفي كل مرة ينشئ سطر `<option>Interest X</option>`.

الخلاصة: الصورة توضح كيف تجعل الصفحة "ذكية"؛ تملأ البيانات وحدها (الإيميل) وتنشئ القوائم وحدها (Loop) بدل الكتابة اليدوية المملة.

ملاحظة ارييسيزية:

لاحظ الرمز <?> المستخدم في الصورة.

هذا اختصار لـ <?php echo >.

يعني بدلاً من كتابة <?php echo \$name?> (طويلة)، نكتب <? name\$ =?> (قصيرة وسريعة).

المصفوفات في PHP (Arrays)

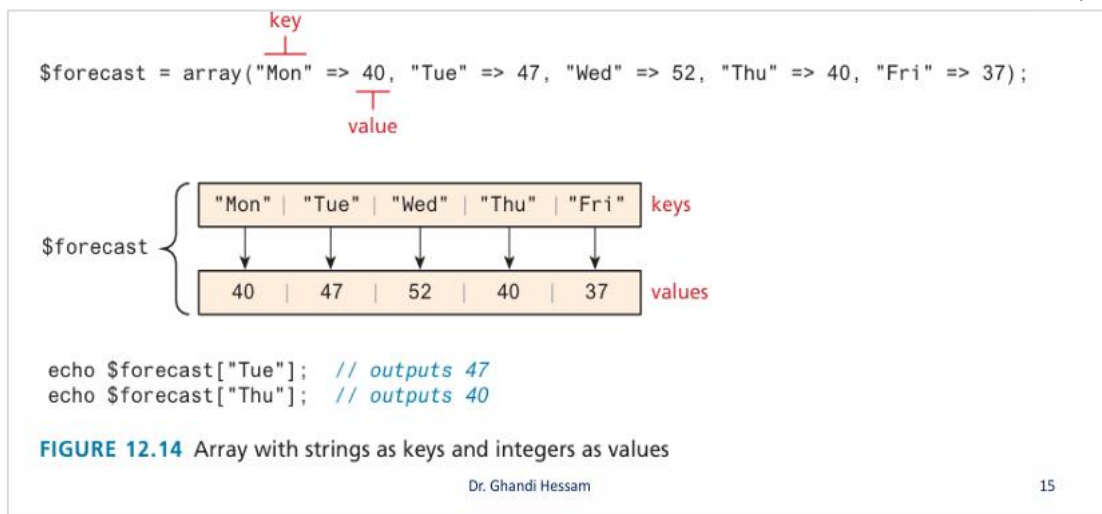
في PHP، أنت قادر على تحديد المفاتيح (Keys) بشكل صريح بالإضافة إلى القيم (Values).

- في اللغات العادية (Indexed Arrays):

- المصفوفة هي قائمة مرقمة آلياً.
- للوصول للعنصر الأول تقول: `list[0]`.
- للوصول للعنصر الثاني تقول: `list[1]`.
- المفتاح (Key) دائماً رقم.

- في PHP (Associative Arrays):

- "أنت لست مجبراً على استخدام الأرقام 0, 1, 2 كمفاتيح".
- يمكنك "تسمية" المفتاح بنفسك.
- **مثال:** بدلاً من أن تقول `user[0]` لتعرف اسم المستخدم، يمكنك أن تقول `user['name']`.
- بدلاً من `user[1]` للعمر، تقول `user['age']`.



المصفوفة في PHP تشبه "القاموس" (Dictionary)؛ كل قيمة لها "اسم" (Key) خاص بها يوصلك إليها، وهذا يجعل الكود أسهل للقراءة والفهم بكثير.

❖ وللمرور على عناصر المصفوفة لدينا طريقتان:

```
// foreach: iterating through the values
foreach ($forecast as $value) {
    echo $value . "<br>";
}
```

1. الطريقة الأولى: الحصول على "القيم" فقط:

○ الكود: `foreach ($forecast as $value)`

- هنا تقول لـ PHP: "خذ مصفوفة \$forecast، ومر على عناصرها واحداً تلو الآخر، وفي كل لفة ضع قيمة العنصر في متغير اسمه \$value".
- متى نستخدمها؟ عندما لا يهمنا ترتيب العنصر أو مفتاحه (Key)، بل نريد فقط البيانات الموجودة بداخله (مثلاً: طباعة قائمة أسماء طلاب فقط).

```
// foreach: iterating through the values AND the keys
foreach ($forecast as $key => $value) {
    echo "day[" . $key . "]=" . $value;
}
```

2. الطريقة الثانية: الحصول على "المفتاح والقيمة" معاً

○ الكود: `foreach ($forecast as $key => $value)`

- هنا الوضع مختلف قليلاً. تقول لـ PHP: "في كل لفة، أعطني اسم المفتاح (ضعه في \$key) وأعطني القيمة (ضعها في \$value)".
- متى نستخدمها؟ عندما يكون المفتاح مهماً. (تذكر السلايد السابق عن المصفوفات التي مفاتيحها أسماء؟).

Outputting a two-dimensional array as a <select> list

```
$stocks = [
    ["AMZN", "Amazon"],
    ["APPL", "Apple"],
    ["MSFT", "Microsoft"]
];

<select>
    <?php
        foreach ($stocks as $s) {
            echo "<option value='$s[0]>'>$s[1]</option>";
        }
    ?>
</select>

<select>
    <option value='AMZN'>Amazon</option>
    <option value='APPL'>Apple</option>
    <option value='MSFT'>Microsoft</option>
</select>
```

result in browser

تحويل بيانات مخزنة في كود (Array) إلى قائمة منسدلة يراها المستخدم (Dropdown Menu).

- البيانات: لديك مصفوفة \$stocks تحتوي على شركات، كل شركة لها "رمز" (مثل AMZN) و"اسم" (مثل Amazon).
- استخدمنا foreach لتدور على هذه الشركات واحدة تلو الأخرى.
- في كل لفة، نأخذ الرمز \$s[0] ونضعه كـ "قيمة خفية" (Value)، ونأخذ الاسم \$s[1] ونضعه كـ "نص ظاهر" في القائمة.

وهكذا الكود يقرأ البيانات ويكتب خيارات القائمة أوتوماتيكياً بدلاً من كتابتها بيدك

:Classes & Object in PHP

يوجد 3 قواعد أساسية لكتابة الكلاسات في PHP:

1. PHP تشبه Java: إذا كنت قد درست Java سابقاً، فستشعر بالراحة هنا. PHP ليست مجرد لغة "سكريبتات" بسيطة، بل هي لغة قوية تعتمد على الـ Classes والـ Objects لبناء أنظمة معقدة.

```
class Artist {
    public $firstName;
    public $lastName;
    public $birthDate;
    public $birthCity;
    public $deathDate;
}
```

```
$picasso = new Artist();
$dali = new Artist();
$picasso->firstName = "Pablo";
$picasso->lastName = "Picasso";
$picasso->birthCity = "Malaga";
$picasso->birthDate = "October 25 1881";
$picasso->deathDate = "April 8 1973";
```

2. دالة البناء (construct):

- هذه الدالة هي أول شيء يعمل أوتوماتيكياً عند إنشاء نسخة (Object) من الكلاس.
- ملاحظة مهمة جداً: الاسم ثابت وهو `construct`. انتبه! توجد شرطتان سفليتان (Double Underscore) قبل كلمة `construct`. نسيان واحدة منهما خطأ شائع جداً.

```
class Artist {
    // variables from previous listing still go here
    ...

    function __construct($firstName, $lastName, $city, $birth,
        $death=null) {
        $this->firstName = $firstName;
        $this->lastName = $lastName;
        $this->birthCity = $city;
        $this->birthDate = $birth;
        $this->deathDate = $death;
    }
}
```

3. كلمة السر (this\$):

- هذا هو الفرق الأكبر عن اللغات الأخرى.
- في لغات أخرى، إذا أردت استخدام متغير داخل الكلاس، تكتب اسمه مباشرة.
- في PHP: ممنوع! يجب أن تسبقه بكلمة this\$-<.
- خطأ: name = "Ali"; (سيعتبره متغيراً محلياً مؤقتاً).
- صح: \$this->name = "Ali"; (هكذا يفهم أنك تقصد المتغير التابع للكلاس).

المصفوفات الشاملة (Superglobal Arrays):

تخيل أن PHP جهزت لك صناديق (مصفوفات) مليئة بالمعلومات جاهزة للاستخدام في أي مكان في الكود (لذلك تسمى Super أو "خارقة" لأنها متاحة في كل مكان، حتى داخل الدوال).

أهم 3 نقاط يجب فهمها هنا:

1. مُعرفة مسبقاً (Predefined): أنت لا تنشئها. هي موجودة وجاهزة بمجرد تشغيل الصفحة.
2. ترابطية (Associative): نصل للبيانات بداخلها عن طريق "الاسم" وليس الرقم.
3. ماذا تحتوي؟ (أمثلة عملية):

- البيانات القادمة من الرابط (URL): تجدها في مصفوفة اسمها \$_GET.
- البيانات القادمة من النماذج المخفية (Forms): تجدها في \$_POST.
- معلومات السيرفر والمتصفح: تجدها في \$_SERVER.

Name	Description
\$GLOBALS	Array for storing data that needs superglobal scope
\$_COOKIES	Array of cookie data passed to page via HTTP request
\$_ENV	Array of server environment data
\$_FILES	Array of file items uploaded to the server
\$_GET	Array of query string data passed to the server via the URL
\$_POST	Array of query string data passed to the server via the HTTP header
\$_REQUEST	Array containing the contents of \$_GET, \$_POST, and \$_COOKIES
\$_SESSION	Array that contains session data
\$_SERVER	Array containing information about the request and the server

• \$GLOBALS

- مصفوفة لتخزين البيانات التي تحتاج إلى نطاق شامل (Superglobal scope).
- هذا "الصندوق الكبير" الذي يحتوي بداخله على كل المتغيرات العامة في البرنامج. يمكنك من خلالها الوصول لأي متغير من أي مكان في الكود.

• COOKIES_\$ (ملاحظة: في PHP القياسية تكتب عادة \$COOKIE بالمفرد)

- مصفوفة لبيانات ملفات تعريف الارتباط (Cookies) التي يتم تمريرها للصفحة عبر طلب HTTP.
- تحتوي على البيانات المخزنة في متصفح المستخدم (مثل "تذكرني" أو تفضيلات اللغة).

• ENV_\$

- مصفوفة لبيانات بيئة الخادم (Server Environment).
- تحتوي على إعدادات خاصة بالسيرفر الذي يعمل عليه الموقع (نادراً ما تستخدمها كمبتدئ).

• FILES_\$

- مصفوفة لعناصر الملفات التي تم رفعها (Uploaded) إلى الخادم.
- أهم مصفوفة عند رفع الصور أو الملفات. عندما يرفع المستخدم صورة، تجد اسمها وحجمها ونوعها داخل هذه المصفوفة.

• GET_\$

- مصفوفة لبيانات "سلسلة الاستعلام" (Query String) التي تمرر للخادم عبر الرابط (URL).
- البيانات الظاهرة في شريط العنوان بالأعلى (مثل: page.php?id=5). تستخدم للبيانات غير الحساسة (مثل البحث).

• POST_\$

- مصفوفة لبيانات "سلسلة الاستعلام" التي تمرر للخادم عبر ترويسة HTTP (HTTP header).
- البيانات المخفية القادمة من النماذج (Forms). تستخدم للبيانات الحساسة (مثل كلمات المرور) لأنها لا تظهر في الرابط.

• REQUEST_\$

- مصفوفة تحتوي على محتويات كل من GET_\$ و POST_\$ و COOKIES_\$.
- هي "الجوكر". إذا كنت لا تعرف هل البيانات قادمة من الرابط (Get) أم من النموذج (Post)، يمكنك البحث عنها هنا لأنها تجمع الكل.

• SESSION_\$

- مصفوفة تحتوي على بيانات الجلسة (Session).
- تستخدم لتذكر المستخدم أثناء تنقله بين صفحات الموقع (مثل بقاء المستخدم "مسجل الدخول" عند انتقاله من الصفحة الرئيسية إلى صفحة الملف الشخصي).

• SERVER_\$

- مصفوفة تحتوي على معلومات حول "الطلب" (Request) والخادم نفسه.
- تحتوي على تفاصيل تقنية، مثل عنوان IP للمستخدم، نوع المتصفح الذي يستخدمه، ومسار الملف الحالي.

مصفوفتا GET_\$ و POST_\$:

الأهمية: تعتبر مصفوفتا GET_\$ و POST_\$ أهم المتغيرات الشاملة (Superglobal Variables) في PHP، لأنهما تسمحان للمبرمج بالوصول إلى البيانات التي أرسلها العميل (المستخدم).

• آلية عمل GET: إذا تم إرسال النموذج (Form) باستخدام طلب HTTP من نوع GET:

- فإن الرابط الناتج (URL) سيحتوي على البيانات ضمن "سلسلة الاستعلام" (Query String).
- ستقوم PHP تلقائياً بملء مصفوفة GET_\$ باستخدام محتويات هذه السلسلة الموجودة في الرابط.

• اما ال POST:

- عندما يرسل المستخدم نموذجاً (Form) وتكون طريقة الإرسال فيه "method=post".
- أين تذهب البيانات؟ عكس ال GET، البيانات هنا لا تظهر في شريط العنوان (URL).
- المسار: البيانات ترسل "مغلقة" داخل ما يسمى جسم طلب (HTTP Request Body).
- في PHP: يقوم السيرفر بفك هذا الغلاف وأخذ البيانات ووضعها تلقائياً في مصفوفة POST_\$.

مقارنة بين GET و POST

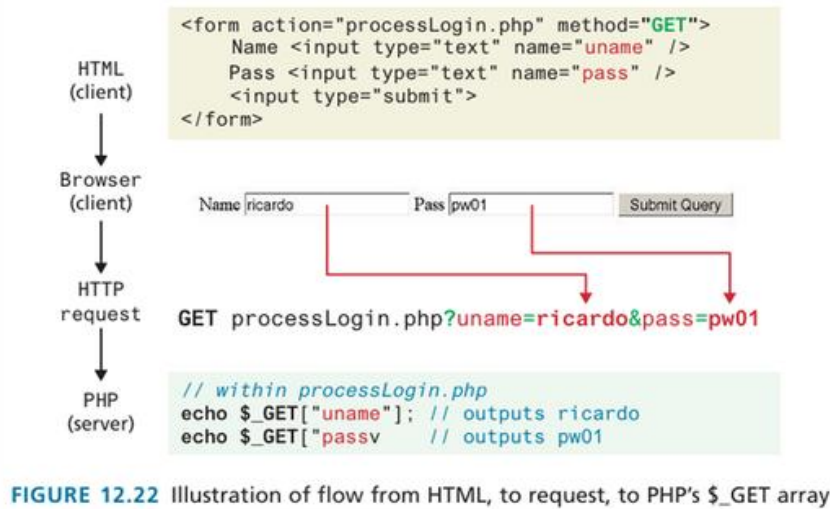
GET (الطريقة المكشوفة)	POST (الطريقة الآمنة/المخفية)
 	 
مرئية في الرابط (URL)  أمان منخفض (مثل بطاقة بريدية)  حجم بيانات محدود (حوالي 2000 حرف)  يمكن حفظها في المفضلة (Bookmark) 	مخفية في جسم الطلب (HTTP Request Body)  أمان عالي (مثل رسالة في ظرف مغلق)  حجم بيانات غير محدود (كبير)  لا يمكن حفظها في المفضلة 
الخلاصة: استخدم GET للبحث والبيانات العامة، واستخدم POST للبيانات الحساسة (مثل كلمات المرور) والملفات الكبيرة.	

قواعد الامان الثلاث:

يجب أن تلتزم بثلاث قواعد صارمة عند التعامل مع كلمات المرور

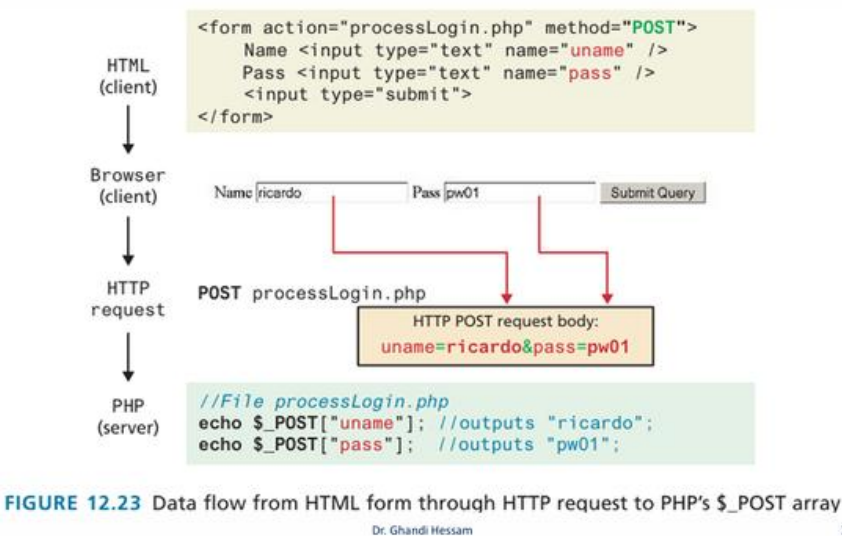
- **استخدم POST حصراً:** لأنك لو استخدمت GET، ستظهر كلمة المرور في رابط الصفحة (URL) ويمكن لأي شخص بجانبك رؤيتها أو حفظها في سجل المتصفح (History).
- **استخدم SSL (القفل الأخضر):** يعني أن يكون رابط موقعك يبدأ ب https://. هذا يقوم بـ "تشفير" البيانات وهي طائرة من جهاز المستخدم إلى السيرفر، فلا يستطيع "الهاكرز" سرقتها في الطريق.

- استخدم `<input type="password">`: هذا يجعل كلمة المرور تظهر كنقاط سوداء (●●●●●) على الشاشة، فلا يراها الشخص الجالس بجانب المستخدم.



Dr. Ghandi Hssam

21



Dr. Ghandi Hssam

22

التأكد من وصول البيانات:

ستمر عليك أوقات أثناء التطوير ب PHP تستخدم فيها نفس الملف للقيام بمهمتين: عرض النموذج (Display Form)، ومعالجة بيانات النموذج (Handle Input). في مثل هذه الحالات، ستحتاج لمعرفة ما إذا كان هناك أي بيانات قد تم إرسالها أصلاً (سواء ب POST أو GET). وللتأكد يوجد خطوتان:

- الخطوة 1: فحص طريقة الطلب (`REQUEST_METHOD`):

يسأل السيرفر: "هل الزائر وصل لهذه الصفحة عن طريق رابط عادي (GET)؟ أم ضغط على زر إرسال (POST)؟"

```
{ ... } if ($_SERVER['REQUEST_METHOD'] == 'POST')
```

إذا كان POST \$ ابدأ معالجة البيانات (تحقق من الباسورد).

إذا كان GET \$ تجاهل المعالجة واعررض النموذج فقط.

• الخطوة 2: فحص وجود البيانات (isset):

داخل الـ POST، نستخدم دالة isset(\$var).

معناها: "Is Set؟" (هل هذا المتغير موجود ومحموز له مكان في الذاكرة؟).

هذا يحميك من الأخطاء لو حاول المستخدم إرسال نموذج فارغ أو ناقص.

```
<!DOCTYPE html>
<html>
<body>
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    if ( isset($_POST["uname"]) && isset($_POST["pass"]) ) {
        // handle the posted data.
        echo "handling user login now ...";
        echo "... here we could redirect or authenticate ";
        echo " and hide login form or something else";
    }
}
?>
<h1>Some page that has a login form</h1>
<form action="samplePage.php" method="POST">
    Name <input type="text" name="uname"><br>
    Pass <input type="password" name="pass"><br>
    <input type="submit">
</form>
</body>
</html>
```

ملاحظة: ماذا اذا كان النموذج فارغاً؟

المستخدم ضغط تسجيل دخول والحقول فارغة، لكن الموقع قبل الطلب! لماذا؟

دالة isset تسأل: "هل المتغير موجود؟"

عندما يرسل المستخدم نموذجاً فارغاً، المتصفح يرسل "user".

بالنسبة لـ isset، المتغير user موجود (له اسم ومكان)، حتى لو كان محتواه فارغاً. فتعطيك "نعم موجود" (True).

الحل (دالة empty):

دالة empty أكثر ذكاءً، فهي تسأل: "هل المتغير غير موجود؟ أو هل هو موجود لكنه فارغ؟"

إذا كان الحقل فارغاً، empty ستعطيك True (نعم هو فارغ).

:Accessing Form Array Data

أحياناً في نماذج HTML، قد يكون لديك قيم متعددة مرتبطة باسم واحد (مثل قائمة تختار منها عدة أيام).

```
<form method="get">
  Please select days of the week you are free.<br>
  Monday <input type="checkbox" name="day" value="Monday"> <br>
  Tuesday <input type="checkbox" name="day" value="Tuesday"> <br>
  Wednesday <input type="checkbox" name="day" value="Wednesday"> <br>
  Thursday <input type="checkbox" name="day" value="Thursday"> <br>
  Friday <input type="checkbox" name="day" value="Friday"> <br>
  <input type="submit" value="Submit">
</form>
```

إذا قام المستخدم باختيار أكثر من يوم وأرسل النموذج، فإن المتغير `$_GET['day']` في المصفوفة الشاملة سيحتوي فقط على آخر قيمة تم اختيارها من القائمة.

❖ **مثلاً:** إذا كان لديك نموذج يحتوي على عدة خيارات (ويمكن اختيار عدة خيارات) فإن المتغير سيحتوي فقط على آخر قيمة قمت باختيارها

• **والحل لذلك:**

يجب عليك تغيير كود HTML في النموذج بالتحديد، سيتوجب عليك تغيير خاصية الاسم (name attribute) لكل مربع اختيار (checkbox) من `day` إلى `day[]`.

```
Monday <input type="checkbox" name="day[]" value="Monday">
Tuesday <input type="checkbox" name="day[]" value="Tuesday">
...
```

• **ولكن ماذا يفعل هذا التغيير؟**

بدلاً من أن يكون المتغير `$_GET['day']` يساوي مجرد كلمة "Monday". يصبح `$_GET['day']` عبارة عن صندوق (مصفوفة) يحتوي على: ["Monday", "Tuesday", "Friday"].

وبالتالي ستخزن الخيارات على شكل مصفوفة ونمر على عناصرها بشكل طبيعي باستخدام `for each`

```
echo "You submitted " . count($_GET['day']) . " values";
foreach ($_GET['day'] as $d) {
    echo $d . " <br>";
}
```

:working with HTTP Header

مفهوم ترويسة (HTTP Header) وعلاقتها بالمخرجات:

لفهم هذه الآلية، يمكننا تشبيه عملية طلب صفحة الويب بإرسال رسالة بريدية. هذه الرسالة تتكون من جزأين رئيسيين:

• **الترويسة (Header) ← تمثل "الظرف الخارجي":**

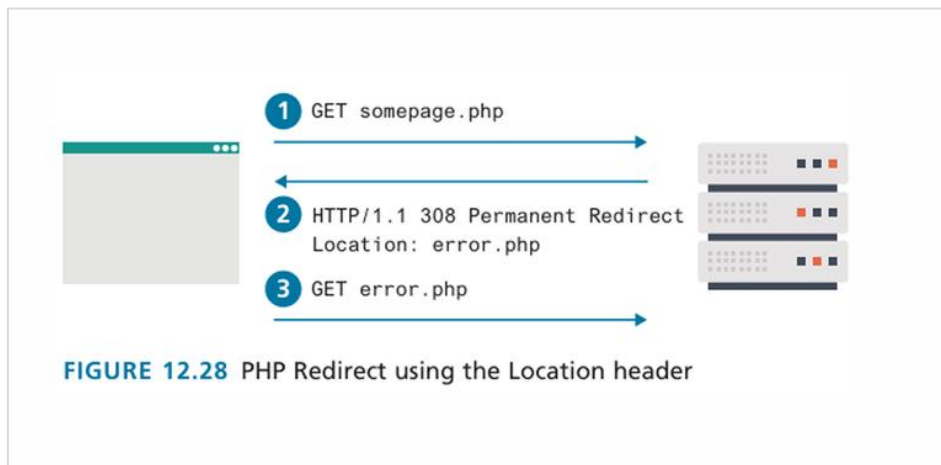
○ تحتوي على معلومات تقنية لا يراها المستخدم العادي (مثل نوع الملف، كود الحالة، توجيه المتصفح لمكان

آخر).

- يتم التعامل معها حصراً عبر دالة header().
- جسم الصفحة (Body) ← يمثل "الورقة داخل الظرف":
 - تحتوي على النصوص، وكود HTML، وكل ما يطبعه المبرمج باستخدام أوامر مثل echo أو print.
 - السيرفر يتبع قاعدة صارمة: "اكتب العنوان على الظرف أولاً، ثم ابدأ بكتابة الرسالة داخل الورقة، وأغلق الظرف فوراً".
 - إذا استخدمت header() أنت تكتب على الظرف (مسموح في البداية).
 - إذا استخدمت echo أنت بدأت الكتابة في الورقة وأغلقت الظرف.
 - **الخطأ:** إذا كتبت في الورقة (echo) ثم حاولت تعديل العنوان على الظرف (header)، سيصرخ السيرفر: "مستحيل! الظرف أغلق وأرسل!".
 - **بالمختصر:** دالة header() يجب استدعاؤها قبل إرسال أي كود (Markup) أو نص إلى المتصفح.

```
<?php
if (! isset($_GET['id'])) {
    header("Location: error.php");
}
...
?>
```

❖ **الاستخدام الشائع:** أحد أكثر الاستخدامات شيوعاً لهذه الدالة هو إعادة توجيه (Redirect) المستخدم من صفحة إلى أخرى.



تخيل أنك ذهبت لمحل (المتصفح) وطلبت غرضاً، فقال لك البائع (السيرفر): "هذا القسم أغلقناه، اذهب للفرع الثاني فوراً" وهذه ما حدث بالصورة السابقة

- المتصفح طلب صفحة somepage.php.
 - السيرفر رد عليه بكود 308 وقال: "غير مسموح لك بالبقاء هنا، اذهب إلى error.php".
 - المتصفح أطاع الأمر فوراً وذهب للصفحة الجديدة.
- وهنا يمكننا رؤية ال redirection الذي تحدثنا عنه سابقاً

:Content-Type HTTP header

ومن اسمها، تُستخدم ترويسة Content-Type لإخبار المتصفح بنوع المحتوى الذي يستقبله في الاستجابة (باستخدام معيار يسمى MIME type).

ملاحظة ارييسيزية:

MIME type (امتدادات بريد الإنترنت متعددة الأغراض) هي معرف قياسي (مثل text/html أو image/png) يُستخدم على الويب لإخبار المتصفح بنوع الملف الذي يستقبله وكيفية التعامل معه أو عرضه، سواء كان نصاً، صورة، فيديو، أو تطبيقاً. تعتمد المتصفحات عليه بدلاً من امتداد الملف لتحديد طريقة المعالجة.

- عادةً، تقوم بيئة PHP بضبط هذه الترويسة تلقائياً على text/html (أي أنها صفحة ويب نصية). ومع ذلك، هناك أوقات قد ترغب فيها بتغيير قيمة هذه الترويسة.
- أحد أكثر الأسباب شيوعاً لفعل ذلك هو عندما تكون صفحة PHP الخاصة بك تعيد بيانات JSON (لتطبيقات الهاتف مثلاً) أو صورة مخصصة (Customized Image).

أمثلة:

1. إذا كنت ترسل بيانات JSON (APIs): يجب أن تخبر المتصفح أن يعامل الكلام كنصوص برمجية وليس صفحة ويب.
2. إذا كنت ترسل صورة (Image): إذا لم تكتب هذا السطر، سيظهر للمستخدم رموز غريبة وعشوائية بدلاً من الصورة.

:JSON (Returning JSON Data)

تستطيع PHP تحويل "المصفوفة الترابطية" (Associative Array) إلى "سلسلة نصية بتنسيق JSON" باستخدام الدالة json_encode().

- PHP تفهم "مصفوفات" (Arrays)، لكن تطبيقات الموبايل أو JavaScript في المتصفح تفهم "JSON".
- هذه الدالة هي "المترجم" الذي يأخذ مصفوفة PHP ويحولها لنص JSON جاهز للإرسال.

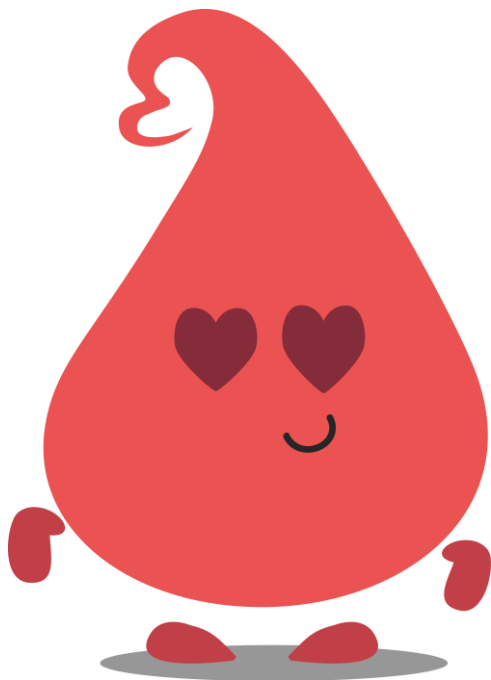
```
<?php
$books = array();

$books[] = ["title"=>"Basics of Web Design", "year"=>2014, "pages"=>400];
$books[] = ["title"=>"Database Processing", "year"=> 2012, "pages"=>630];
$books[] = ["title"=>"Development Economics", "year"=>2014, "pages" => 760];

header('Content-Type: application/json');
echo json_encode($books, JSON_NUMERIC_CHECK);
?>
```

المعامل JSON_NUMERIC_CHECK يخبر الدالة بأن تقوم بترميز القيم الرقمية بدون علامات تنصيص (بدون Quotes).

- أحياناً تكون الأرقام في قاعدة البيانات مخزنة كنصوص (مثلاً السعر "50").
- عند تحويلها لـ JSON، ستظهر هكذا: "price": "50" (بين علامتي تنصيص).
- المشكلة: JavaScript ستتعامل معها كـ "كلمة" وليس "رقم"، مما قد يسبب مشاكل في العمليات الحسابية.
- **الحل:** نضع هذا المعامل JSON_NUMERIC_CHECK ليقوم PHP بإزالة العلامات عن أي شيء يشبه الرقم.
- **مثال:** "pages": "760" (رقم).
- **بدلاً من:** "pages": "760" (نص).



وإلى هنا تكون قد انتهت محاضرتنا
نلتقاكم في المحاضرات القادمة...

انتهت المحاضرة

<The end>